

Where we are standing

A backtester that reports an estimate, its error bar, and the probability the number survived the search that produced it.

Commit	6a3b5a7 on main · 15 commits	Suite	117 collected · 113 pass · 4 skip · ~40 s
Latest	feat(gates): G2 twin-engine agreement, exact	CI	green in 43 s · 5 checks · offline
Code	819 package · 1,477 test · 84 script LOC	Quality	ruff clean · mypy --strict clean (22 files)

Four sessions in. Gate 0.0, G1, G7 and G2 are green, the engine core is certified and still entirely offline, and no market data has entered the system — invariant B1 holds. The headline result of the session is that **G2 came out exact** rather than merely inside tolerance: two independently written engines agree bitwise.

Gate status

Gate	Statement	Status	Evidence
0.0	Reproduce the propositions first	GREEN	E[max z] to 0.58 SE; Prop 3 slope -0.9989 ± 0.0014
G1	Causality: scramble future, past bit-identical	GREEN	500 cuts × 20 seeds per strategy, 3 strategies
G2	Twin engines agree to 1e-12	GREEN	0.000e+00 across zoo × 3 conventions × cost sweep
G3	Analytic recovery on synthetic GBM	not started	next session
G4	Zero-cost identity	partial	exact float equality for buy-and-hold, all conventions
G5	Cost monotonicity, break-even cost	not started	next session
G6	Null calibration, 1,000 coin flips	not started	needs the data layer first
G7	Leakage trap must fire	GREEN	4 of 02 A3's classes rejected; oracle case resolved
G8	Purged, embargoed walk-forward	not started	deferrable per 03 Part C
G9	PBO via CSCV	partial	SelectionRule interface built first, by design
G10	Reproducibility from pinned hashes	partial	figure bytes + numeric output stable across runs

G2 — the session's result

Two engines, written independently, sharing exactly one thing: the warm-up index. That is shared deliberately, because an off-by-one in the warm-up is a specification question rather than an implementation one, and duplicating it would let both engines be wrong in the same direction while agreeing perfectly. All of the Part E arithmetic is written twice on purpose — two engines that share their accounting agree by construction and certify nothing.

Measurement	Value	Reading
Max relative equity deviation	0.000e+00	Exact, against a 1e-12 requirement
Coverage	3 strategies × 3 conventions × 5 cost models	Plus hypothesis over arbitrary price series
Planted engine disagreement	7.417e-02	Gate discriminates; it is not passing vacuously
Convention spread (final equity)	12924.37 / 12915.83 / 11688.41	close_to_close / next_open / next_close

Exact agreement is legitimate here rather than suspicious: the event engine's hard-sliced signal window contains the same elements as the vectorised engine's rolling window, and the arithmetic order coincides. The planted-bug test is what proves the harness can fail. The equity path is an explicit loop in *both* engines because it is not a cumprod — cost[t] depends on equity[t-1], and Part F2's approximate route carries error of order cost² that would never survive 1e-12.

What the gates caught

Every row is a real defect in code I had written, found by a check rather than by reading. This is the argument for the whole approach, so it is worth keeping the score honestly.

Found by	Defect	Why it was dangerous
hypothesis	Constant column escaped the degeneracy check	60 copies of 0.001 give $\text{std} \approx 2\text{e-}19$, not 0.0, so an equality test passed it through and the Sharpe came out near 1e16 — enough to take the entire portfolio under ArgMax or Softmax
hypothesis	Softmax amplified rounding dust into a decision	Columns tied on true Sharpe leave $\sim 1\text{e-}17$ of float noise; z-scoring divides by it, and at $\tau=0.25$ that became a 0.9997 / 0.0003 allocation. CSCV blocks produce near-identical Sharpes routinely, so G9's temperature sweep would have been noise wherever the grid agrees
ruff B905	$8 \times \text{zip}()$ without <code>strict=</code>	Silent truncation on a length mismatch — a wrong statistic and no exception
mypy --strict	18 typing gaps, 11 of them bare dict returns	Drove the switch to frozen dataclasses (B7): one state per result object, which is what makes the twin-engine comparison meaningful
CI	Unresolvable action tag setup-uv@v9	A run that dies in 3 s having executed no step. Nothing in the repo could detect it, so a resolver now runs as the first check
CI	Local-green, CI-red property tests	hypothesis switches to a derandomised <i>ci</i> profile, drawing a different sequence. Fixed by constructing valid inputs instead of filtering for them

Decisions taken

Decision	Rationale
diff = 0.00e+00 is the single standard	Experiment A is judged against $E[\max z]$ computed by quadrature and checked to machine precision against $1/\sqrt{\pi}$ and $3/(2\sqrt{\pi})$, not against the two-term Gumbel approximation. The empirical mean estimates the truth; the formula only approximates it. Comparing Monte Carlo to the approximation conflated the two and forced an arbitrary small-N exclusion. Against exact truth it holds at every N from 2 to 10■, worst deviation 1.98 SE.
Accumulated error bands retired	Trialled and withdrawn on evidence. The pooled statistics behaved as theory predicts ($\text{rms}_z = 1.0959$ in $[0.4117, 1.5883]$) and the band did discriminate, driving a biased reference to $\text{rms}_z = 20.83$. But $\text{mean} \text{diff} $ came out at $8.87\text{e-}04$ — below the proposed lower bound of 0.0025 — and it scales as $1/\sqrt{\text{reps}}$, so any fixed bound on it constrains compute budget rather than correctness. Recoverable: <code>git checkout 37d4dac -- falsify/accumulation.py</code>
Monte Carlo reduced, never removed	Still 3 of the 5 Experiment A tests and $\sim 2.6\text{M}$ draws — more sampling than before the fix. An exact reference says where the truth is; only sampling says whether the estimator finds it.
02 Part A4's oracle trap resolved	A4 asserts G1 must catch LeakyOracle. It provably cannot: <code>close[t]</code> lies inside <code>bars[0:t+1]</code> , which the Part A1 contract permits, and is never scrambled for $t \leq \tau$. The strategy is legal and the one-bar lag is the engine's job. The harness now takes both cuts and asserts both behaviours, so the discrepancy is a property of the code rather than a claim in a comment. Open for your ruling if you would rather tighten A1.
Actions pinned to exact releases	No floating majors anywhere. A project asserting byte-identical output under G10 should not let its own CI toolchain drift underneath it. A test enforces it.

Error budget of SR

The two-term Gumbel expression is what 01 Part B3 feeds the Deflated Sharpe as the benchmark SR, so its distance from exact truth is a bias in the DSR itself. The sign matters: it **overstates for every $N \geq 3$** , so SR is too strict rather than too lax and the resulting DSR is conservative. Below $N = 100$ the error exceeds 1% and should be quoted alongside any DSR computed there. All of it is asserted, not assumed.

N	2	3	5	10	50	100	1,000	10
Error in SR	-7.88%	+0.77%	+2.55%	+2.33%	+1.21%	+0.92%	+0.42%	+0.10%

The error is non-monotone at the bottom: the signed error flips sign between $N = 2$ and $N = 3$, so a naive monotone-decrease assertion would have been false. It decays monotonically from $N = 5$ upward.

What exists now

Module	Role	Tests
falsify/core/causality.py	G1 harness, two cut modes	14
falsify/core/event.py	Reference engine — hard-sliced prefixes, $O(T^2)$	33
falsify/core/vectorized.py	Product engine — rolling ops, exact equity loop	(shared)
falsify/core/types.py	Bars, Result, slice, InsufficientHistory — all frozen	(shared)
falsify/core/conventions.py	Part D's three conventions as (price, lag)	(shared)
falsify/costs.py	Commission, spread, slippage, borrow, cash yield	(shared)
falsify/selection.py	ArgMax, Softmax, EqualWeight, TopK	52
falsify/strategies/base.py	Strategy ABC — target weights, never orders (B4)	(shared)
tests/gates/test_prop.py	Gate 0.0 — Experiments A, B, C + the figure	14
scripts/check_action_refs.py	Resolves every workflow uses: ref	4

Next session

Per 03 Part C: **G3 + G4 + G5 together**, all three small. G3 is genuinely easy now that the synthetic generators and both engines exist — generate GBM at known μ and σ , run buy-and-hold through the engine, and confirm the estimated annualised Sharpe recovers $(\mu - \sigma^2/2)/\sigma = 0.30$ within 2 SE over 200 paths. G4 is one assertion beyond what already passes. G5 is a cost sweep plus a $\text{np.diff}(...) \leq 0$ check, and it yields break-even cost — the single most useful number in the eventual report.

That completes the certified offline core. **The first network call comes after it**, and not before: B1 holds until Gate 0 is green.

Open question for you

02 Part A4 versus Part A1 — whether to tighten the causality contract so that a signal at bar t may use only bars[0:t] strictly. Tightening it would make LeakyOracle a G1 failure as A4 intends, but it would also fail any legitimate close_to_close strategy, which Part D explicitly permits. Both readings are currently implemented and asserted; the default is Part A1. One line changes it.

Generated 2026-08-08 from 6a3b5a7 · every number measured on this tree, not recalled · regenerate with: `uv run --group docs python scripts/status_report.py`